

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería en Software

SGED

Sistema de Gestión para la
Escuela Deportiva ProFútbol

Informe de la Tercera Entrega

Proyecto Fin de Curso — Aplicaciones Web

Asignatura: Aplicaciones Web (Quinto nivel)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Periodo: PPA 2026-2027

Versión: v0.9.0-rc (candidata)

Integrantes:

Arcalle Grefa Darwin Orlando

Pallo Pinto Alejandro Daniel

Velez Lopez Ricardo Elias

Repositorio:

https://github.com/DarwinSM21/SGED_APPWEB

Quevedo — Los Ríos — Ecuador — 2026

Índice

1. Introducción	2
1.1. Estado de la entrega: declaración previa	2
1.2. Pila tecnológica	2
2. Resolución de las observaciones previas (Bloque 0)	2
3. Reestructuración de paquetes y reconciliación de documentación	4
3.1. Los paquetes reservados vacíos: por qué se eliminaron	4
4. Arquitectura	5
4.1. Vista de contenedores	5
4.2. Autenticación: JWT en cookie <code>HttpOnly</code>	5
5. Estrategia híbrida de acceso a datos	5
5.1. Un defecto real: <code>FUNCTION</code> frente a <code>PROCEDURE</code>	6
5.2. Ausencia de SQL dinámico	6
6. Requisitos y modelo de datos	6
6.1. Resumen de requisitos	6
6.2. Modelo de datos	7
7. Evidencia empírica	8
7.1. Rendimiento (Bloque C.1)	8
7.2. Seguridad: OWASP Top 10 (Bloque C.4)	8
7.3. Cobertura de pruebas	9
7.4. Calidad web: Lighthouse (Bloque C.5)	10
7.4.1. Por qué el SEO de 63 es el resultado correcto	10
7.5. Usabilidad: SUS (Bloque C.3)	11
8. Reproducibilidad	11
9. Consideraciones éticas	12
10. Trazabilidad y honestidad académica	12
11. Conclusiones	12

1. Introducción

Este informe documenta la Tercera Entrega del Proyecto Fin de Curso de la asignatura Aplicaciones Web: el sistema **SGED**, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol.

El eje de esta entrega es la *verificabilidad*. Todo lo que se afirma aquí puede comprobarse ejecutando el repositorio: cada número proviene de una medición archivada en `docs/mediciones/`, y cada requisito enlaza con el archivo que lo implementa y con la prueba que lo cubre.

1.1. Estado de la entrega: declaración previa

Antes de exponer los resultados conviene ser explícito sobre dos cosas:

1. **Alcance realmente implementado.** El sistema expone hoy 13 *endpoints* REST (autenticación y gestión de estudiantes). El dominio deportivo (entrenadores, horarios, sesiones, asistencias y evaluaciones) está *modelado y migrado* en la base de datos, pero no expuesto vía API. En el documento de requisitos cada elemento indica su estado real, y en este informe se mantiene la misma distinción. No se presenta como entregado nada que no lo esté.
2. **Medición de usabilidad completada, con un resultado que no se suaviza.** La encuesta SUS (Bloque C.3, sección 7.5) se aplicó a 10 participantes reales: media SUS 68,25, apenas por encima del umbral de 68, y con un patrón bimodal marcado por perfil que la media agregada esconde.

1.2. Pila tecnológica

Capa	Tecnología
Backend	Spring Boot 3.2 (Java 21 LTS), Spring Data JPA, Spring Security
Frontend	Angular, servido por nginx con terminación TLS
Base de datos	PostgreSQL 16 (esquemas seguridad y deportivo)
Caché y revocación	Redis 7
Migraciones	Flyway (V1–V6)
Orquestación	Docker Compose, imágenes fijadas por digest SHA-256

2. Resolución de las observaciones previas (Bloque 0)

El docente emitió doce observaciones sobre las entregas 1A y 1B. Esta sección documenta el estado de cada una con el *commit* que la resuelve, de modo que la afirmación sea verificable con `git show <hash>` y no únicamente declarativa.

Cód.	Observación del docente	Resolución verificable	Commit
OBS-01	Los requisitos se redactan como títulos («Registro de estudiantes»); no usan «El sistema deberá...».	<code>docs/requisitos/SRS.md</code> reescribe los 22 RF y 13 RNF con la forma normativa y criterio de verificación [7].	98bab0d
OBS-02	Los diagramas C4 N1/N2 y el MER contienen texto marcador «[Reemplazar con PNG]».	<code>docs/diagramas/</code> contiene los PNG reales más las fuentes <code>.puml</code> y <code>.dbml</code> .	bf6ee80

Cód.	Observación	Resolución	Commit
OBS-03	Incoherencia de pila: el ADR decide PHP/Laravel/MySQL, pero el repositorio construye Spring Boot/Angular/PostgreSQL.	ADR-001 reescrito: evalúa las tres opciones y decide explícitamente Java 21 con Spring Boot 3.2.5, coherente con el código.	bf6ee80
OBS-04	<code>query.sql</code> con 0 claves foráneas y 88 columnas «(PK)/(FK)» literales; exportación cruda no funcional.	<code>db/schema.sql</code> declara 21 restricciones de integridad referencial reales.	db77ce2
OBS-05	Roles sin completar («Integrante 1/2/3»); wireframes con marca ajena; falta <code>.env.example</code> .	<code>CONTRIBUTORS.md</code> asigna roles CRediT por evidencia de <code>git log</code> ; <code>.env.example</code> versionado.	a8d6f18
OBS-06	En el repositorio solo consta ADR-003; los diagramas aparecen en el informe como texto.	Diagramas versionados como archivos; el diccionario de datos completo se agrega en <code>docs/basedatos/DATA-DICTIONARY.md</code> .	98bab0d
OBS-07	<code>AuthController</code> solo expone <code>/login</code> , <code>/me</code> y <code>/ping</code> ; no hay registro, logout ni refresh. <code>RedisBlacklistService</code> existe pero no se invoca.	Seis <i>endpoints</i> operativos; el logout invoca realmente la lista de revocación por <code>jti</code> .	b907d10, bcb3642
OBS-08	La migración <code>V1</code> está vacía (0 bytes) y <code>V2</code> depende de un esquema que ninguna migración crea; con <code>ddl-auto=validate</code> el arranque no es reproducible.	<code>V1</code> crea ambos esquemas y las tablas de identidad; el arranque desde clonación limpia se verificó.	b907d10
OBS-09	La lista de revocación no está cableada a ningún <i>endpoint</i> y no se aplica <code>@PreAuthorize</code> .	Ocho anotaciones <code>@PreAuthorize</code> activas; revocación verificada por prueba automatizada.	688c4be, 90c6c57
OBS-10	<code>AuthServiceTest</code> está vacío (0 bytes); la única prueba real es <code>contextLoads()</code> . No se cumple el mínimo de 5 pruebas. La colección Postman no está versionada.	34 pruebas en 7 clases; cobertura real 68,1 %; <code>docs/postman/coleccion.json</code> versionada.	1e798bc
OBS-11	<code>docker-compose.yml</code> está vacío (0 bytes); no hay orquestación verificable.	Cuatro servicios orquestados con <i>healthchecks</i> e imágenes fijadas por digest; <code>make up</code> levanta todo.	b907d10, 17dc5df
OBS-12	Varios contenidos del informe no se corresponden con lo presente en el repositorio (están vacíos o ausentes).	Cada requisito declara su estado real (implementado / modelado / planificado); esta entrega no afirma nada no verificable.	98bab0d

Las doce observaciones tienen resolución trazable. La más costosa de resolver no fue una funcionalidad ausente sino OBS-12, porque exigió revisar sistemáticamente qué afirmaba el informe anterior frente a lo que el repositorio realmente contenía.

3. Reestructuración de paquetes y reconciliación de documentación

Entre la primera versión de este informe y esta revisión, el equipo reestructuró el backend en tres dominios (`academico`, `deportivo`, `seguridad`) y normalizó la categoría del estudiante: dejó de ser un texto libre validado por patrón (`VARCHAR`, `SUB-NN`) para convertirse en una entidad propia (`deportivo.categorias`) referenciada por clave foránea. Aparecieron además cinco recursos CRUD nuevos (`Categoria`, `Entrenador`, `Usuario`, `Persona`, `EstadoGeneral`) que no tenían requisito documentado, y dos paquetes reservados sin contenido (`Representante`, `Equipo` — clases vacías, sin tabla en el esquema) que después se eliminaron por las razones que se explican más abajo.

Este informe, el SRS (`docs/requisitos/SRS.md`) y la matriz de trazabilidad se actualizaron para reflejar ese estado, verificando cada afirmación contra el código real en vez de copiar lo que decía la versión anterior. Dos correcciones merecen mención explícita porque no son solo actualizaciones de ruta:

- **Cobertura de pruebas.** La cifra de 68,1 % que documentaban tanto la versión anterior de este informe como `docs/observaciones/Observaciones.md` era correcta *en el momento en que se midió*. Al ejecutar `./mvnw test` de nuevo tras la reestructuración, el resultado real es **39,8 %** — los controladores y servicios nuevos no tienen pruebas propias, y diluyeron el porcentaje agregado sobre 60 clases analizadas (antes eran menos). Es una regresión real por debajo del umbral del 60 % exigido, no un error de medición, y se reporta como tal en la sección 7.3 en vez de repetir la cifra anterior.
- **Datos de salud sin resolución ética.** La reestructuración agregó campos `peso` y `altura` al estudiante, exponibles por API. `docs/etica/ETHICS.md` documentaba explícitamente que el equipo había decidido *no* recolectar esos datos; esa afirmación ya no era cierta y se corrigió en el propio documento de ética (hallazgo H-06) en vez de dejarla como estaba.

3.1. Los paquetes reservados vacíos: por qué se eliminaron

Los dos paquetes reservados merecen una decisión explícita y no un comentario al margen. `academico.representante` y `deportivo.equipo` contenían catorce clases —controlador, servicio, repositorio, entidad y DTO de cada módulo— con la declaración de paquete, la firma de la clase y el cuerpo vacío. Una de ellas, `RepresentanteController.java`, era literalmente un archivo de **cero bytes**.

Ese detalle no es cosmético en el contexto de este proyecto. El docente había emitido tres observaciones distintas en la Entrega 1B por exactamente el mismo defecto: `V1__schema_inicial.sql` vacía (OBS-08), `AuthServiceTest.java` vacío (OBS-10) y `docker-compose.yml` vacío (OBS-11). Conservar un archivo de cero bytes después de que ese patrón ya fuera observado tres veces habría sido repetir un defecto conocido, y además tenía un efecto medible: las clases vacías aportaban constructores por defecto no cubiertos que contaban como instrucciones perdidas en JaCoCo.

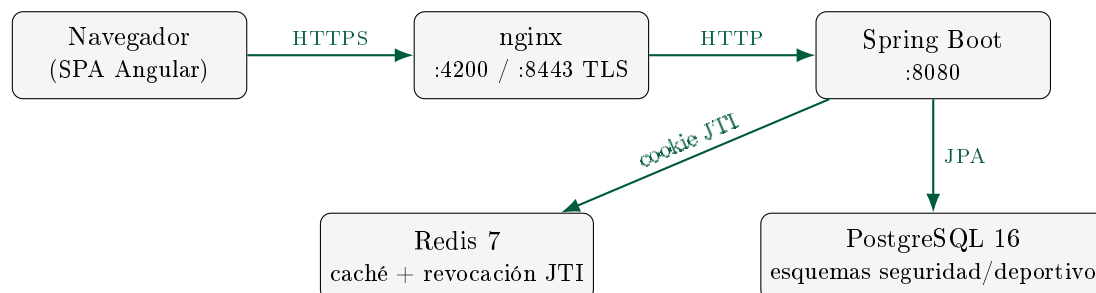
Había dos salidas honestas: implementar los dos módulos, o eliminarlos. Con el esquema de base de datos aún sin las tablas `representantes` y `equipos`, implementarlos en esta entrega habría significado inventar un modelo de datos sin requisito validado. Se optó por eliminar las catorce clases —ninguna estaba referenciada desde el resto del código, lo que confirma que no eran un punto de extensión en uso— y dejar ambos módulos declarados como pendientes únicamente en la documentación (RF-22 en el SRS, la matriz de trazabilidad y este informe). La diferencia es relevante para la evaluación: un módulo pendiente documentado es planificación; un paquete de clases vacías en el repositorio aparenta una implementación que no existe.

También se corrigió una inconsistencia en ADR-002 (autenticación): describía JWT almacenado en `localStorage` con un interceptor agregando `Authorization: Bearer`, cuando el código

real —y el ADR-007 de este mismo informe— usa cookies `HttpOnly`. Es el mismo patrón que ya había observado el docente en la Entrega 1A (OBS-03: un ADR describía PHP/Laravel mientras el repositorio construía Spring Boot). Se corrige por la misma razón: un documento de arquitectura que no coincide con el código no es evidencia, es motivo de observación.

4. Arquitectura

4.1. Vista de contenedores



La documentación de arquitectura en el modelo C4 [5] y las decisiones arquitectónicas (ADR) [10] se mantienen en `docs/diagramas/` y `docs/adr/`.

4.2. Autenticación: JWT en cookie `HttpOnly`

El sistema emite el JWT [8] exclusivamente en una cookie `HttpOnly`, `Secure` y `SameSite=Strict`. El token nunca viaja en el cuerpo de la respuesta ni se almacena en `localStorage`, de modo que un ataque de *scripting* entre sitios no puede leerlo.

```

1 ResponseCookie.from("sged_access", token)
2   .httpOnly(true)
3   .secure(cookieSecure)
4   .sameSite("Strict")
5   .path("/")
6   .maxAge(Duration.ofMillis(expiracionMs))
7   .build();

```

Listing 1: Emisión de la cookie de sesión (`AuthController.java`)

Como JWT es *stateless*, cerrar sesión no invalidaría el token por sí solo. Por eso el sistema mantiene una lista de revocación en Redis indexada por el identificador del token (`jti`), con tiempo de vida igual al tiempo que le restaba al token: un token cerrado deja de ser aceptado inmediatamente, aunque su firma siga siendo válida.

5. Estrategia híbrida de acceso a datos

El Bloque A.2 de la guía exige una separación explícita entre lo que resuelve el ORM y lo que debe resolver el motor de base de datos. La regla adoptada es:

Spring Data JPA (ORM)	Procedimientos almacenados
CRUD elemental, consultas paginadas, búsquedas por identificador.	Agregaciones, actualizaciones masivas con criterio de negocio, operaciones multi-tabla transaccionales.

Los dos procedimientos implementados están versionados en `db/procs/` y creados por migración Flyway [13]:

Procedimiento	Requisito	Motivo
sp_contar_estudiantes_activos	RF-14	Agregación COUNT
sp_desactivar_estudiantes_categoria	RF-15	Actualización masiva transaccional

5.1. Un defecto real: FUNCTION frente a PROCEDURE

La migración V5 creó estos objetos como FUNCTION. Al invocarlos desde Java con `@Procedure`, Spring Data usa `CallableStatement` con sintaxis `{call ...}`, y PostgreSQL solo acepta `CALL` contra procedimientos reales [12]; el error era explícito: «... is not a procedure. Hint: To call a function, use `SELECT`.»

La migración V6 los convierte en PROCEDURE con parámetro de salida. Se documenta porque ilustra la diferencia entre «el procedimiento existe» y «el procedimiento se invoca de verdad»: en la versión anterior el objeto SQL estaba creado pero quedaba huérfano, y la aplicación seguía contando en memoria.

```

1 @Procedure(procedureName = "academico.sp_contar_estudiantes_activos")
2 Long contarEstudiantesActivosPorCategoria(@Param("p_categoria") Long idCategoria
    );

```

Listing 2: Invocación real desde el repositorio (academico/estudiante/repository/EstudianteRepository.java)

Nota sobre esta segunda versión del informe: el procedimiento se movió del esquema `seguridad` a `academico` y su parámetro cambió de `VARCHAR` (nombre de categoría) a `Long` (`id_categoria`) cuando la categoría se normalizó como entidad propia — ver sección 3.

5.2. Ausencia de SQL dinámico

Ninguna capa construye sentencias SQL por concatenación de cadenas. Toda consulta usa parámetros vinculados (JPA) o parámetros nombrados (los procedimientos). El repositorio incluye una auditoría automatizada (`scripts/audit-sql-dynamic.sh`) integrada en `make audit`.

6. Requisitos y modelo de datos

6.1. Resumen de requisitos

La especificación completa está en `docs/requisitos/SRS.md`. Se resumen aquí los requisitos funcionales con su estado real y el *endpoint* o esquema que los materializa.

ID	Requisito	Implementación	Estado
RF-01	Registro de usuarios por administrador	POST /api/auth/registro	OK
RF-02	Autenticación con JWT en cookie <code>HttpOnly</code>	POST /api/auth/login	OK
RF-03	Cierre de sesión con revocación efectiva por <code>jti</code>	POST /api/auth/logout	OK
RF-04	Renovación de sesión sin re-credenciales	POST /api/auth/refresh	OK
RF-05	Consulta de la sesión activa	GET /api/auth/me	OK
RF-06	Bloqueo tras 5 intentos fallidos en 15 minutos	LoginAttemptService	OK
RF-07	Comprobación de disponibilidad	GET /api/auth/ping	OK
RF-08	Listado paginado de estudiantes	GET /api/estudiantes	OK
RF-09	Consulta por identificador con 404 tipificado	GET /api/estudiantes/{id}	OK

ID	Requisito	Implementación	Estado
RF-10	Registro transaccional de estudiante	POST /api/estudiantes	OK
RF-11	Validación del formato de categoría (SUB-NN)	EstudianteRequest	OK
RF-12	Actualización de estudiante	PUT /api/estudiantes/{id}	OK
RF-13	Baja lógica preservando el historial	DELETE /api/estudiantes/{id}	OK
RF-14	Conteo de activos por categoría (en el motor)	sp_contar_estudiantes_activos	OK
RF-15	Desactivación masiva por categoría (en el motor)	sp_desactivar_estudiantes_categoria	OK
RF-16	Gestión de entrenadores	deportivo.entrenadores	Modelado
RF-17	Horarios recurrentes de entrenamiento	horarios_entrenamiento	Modelado
RF-18	Sesiones de entrenamiento con ciclo de estados	sesiones_entrenamiento	Modelado
RF-19	Asistencia por RFID o manual	deportivo.asistencias	Modelado
RF-20	Evaluación diaria por criterios	detalle_evaluacion	Modelado
RF-21	Promedios de evaluación	v_promedio_evaluacion	Modelado
RF-22	Notificación a representantes	—	Planificado

Los requisitos no funcionales (RNF-01 a RNF-13) se verifican con las mediciones de la sección siguiente. La matriz `docs/trazabilidad/matriz.csv` enlaza cada uno con su prueba y su evidencia.

6.2. Modelo de datos

El esquema se divide en dos espacios de nombres con responsabilidades separadas:

Esquema	Tablas	Contenido
seguridad	6	Personas, usuarios, roles, relación usuario-rol, estados y estudiantes.
deportivo	9	Posiciones, entrenadores, horarios, sesiones, asistencias, criterios, evaluaciones, detalle y observaciones.

Decisiones de modelado relevantes, todas materializadas como restricciones en el motor y no solo como validación en la aplicación:

- **Baja lógica en lugar de borrado.** Las asistencias y evaluaciones referencian al estudiante por clave foránea; borrarlo físicamente destruiría su historial deportivo.
- **Unicidad parcial del código RFID.** `UNIQUE ...WHERE rfid_codigo IS NOT NULL` permite muchos estudiantes sin credencial, pero impide que dos compartan la misma.
- **Una asistencia por sesión y estudiante.** `UNIQUE (id_sesion, id_estudiante)`.
- **Posición jugada por evaluación.** `detalle_evaluacion` guarda la posición de *ese* día, distinta de la habitual del estudiante, para que el histórico quede correctamente etiquetado cuando alguien juega fuera de su posición.
- **Coherencia temporal.** `CHECK (hora_fin > hora_inicio)` y `CHECK (dia_semana BETWEEN 1 AND 7)` en los horarios.

El diccionario completo, con tipos, restricciones y disparadores, está en `docs/basedatos/DATA-DICTIONARY.md`. El esquema no lo genera el ORM: `ddl-auto` está fijado en `validate`, de modo que la aplicación falla al arrancar si el esquema real y las entidades divergen.

7. Evidencia empírica

Todas las mediciones de esta sección se generaron ejecutando el sistema real levantado con `make up`, no en simulación, y sus artefactos crudos están versionados.

7.1. Rendimiento (Bloque C.1)

Tres corridas independientes de k6 con 50 usuarios virtuales durante 30 segundos contra el *endpoint* protegido `GET /api/estudiantes`:

Corrida	Media (ms)	Mediana	p90	p95	RPS
1	6,53	5,26	11,38	14,45	404,21
2	6,67	5,94	11,63	15,32	403,52
3	6,25	5,66	10,37	12,78	405,99
Agregado	6,48	—	—	14,18	404,57

Con intervalo de confianza al 95 %: media $6,48 \pm 0,53$ ms, p95 $14,18 \pm 3,20$ ms, throughput $404,57 \pm 3,16$ RPS, y **0 % de errores** en las tres corridas.

El umbral exigido es $p95 < 200$ ms con caché caliente [6]. El resultado lo cumple con un margen de más de un orden de magnitud. **OK**

7.2. Seguridad: OWASP Top 10 (Bloque C.4)

Seis controles verificados con peticiones reales contra el sistema en ejecución [11]:

Control	Resultado	Evidencia observada
A01 Control de acceso	OK	403 al operar sin el rol requerido
A02 Fallos criptográficos	OK	TLS 1.3 en :8443; BCrypt coste 12
A03 Inyección	OK	422 ante entrada malformada; sin SQL dinámico
A05 Configuración	OK	CSP, HSTS, nosniff, X-Frame-Options: DENY
A07 Fallos de autenticación	OK	Bloqueo exacto en el sexto intento (429)
A09 Registro y monitoreo	OK	AUTH_LOGIN_OK/FAIL con IP, marca de tiempo y sujeto

Los errores se devuelven como `ProblemDetail` [9], sin trazas de pila ni detalles internos. Ejemplo real capturado para A07:

```

1 HTTP/1.1 429
2 Content-Type: application/problem+json
3 {
4   "type": "https://sged.uteq.edu.ec/errores/DemasiadasPeticiones",
5   "title": "Too Many Requests",
6   "status": 429,
7   "detail": "Demasiados intentos fallidos. Intente mas tarde."
8 }
```

Listing 3: Respuesta al sexto intento de autenticación

7.3. Cobertura de pruebas

Medido el 2026-07-30 con construcción limpia (`./mvnw clean test`): **72,7 % de cobertura de instrucciones (2466 cubiertas de 3393). Cumple el umbral del 60 %**. El camino hasta este número quedó documentado en vez de solo reportar el resultado final: al medir por primera vez después de la reestructuración de paquetes (sección 3) el resultado real era **39,8 %** — los recursos nuevos (`Categoria`, `Entrenador`, `Usuario`, `Persona`, `EstadoGeneral`) no tenían ninguna prueba propia. Se agregaron 57 pruebas nuevas para esos cinco recursos (10 clases: servicio y controlador de cada uno), y la cobertura subió a **72,7 %**.

La palabra «limpia» está ahí por una razón concreta. La primera medición posterior a la reestructuración se hizo con `./mvnw test` sobre un directorio `target/` que todavía conservaba archivos `.class` compilados *antes* del cambio de paquetes. El reporte que se llegó a archivar como evidencia listaba paquetes que ya no existen en el código fuente (`org.uteq.backend.auth.*`, `org.uteq.backend.estudiante.*`): JaCoCo instrumenta lo que encuentra en `target/classes`, no lo que hay en `src/`. La cifra publicada aquí proviene de `clean test` y el reporte archivado en `docs/mediciones/jacoco/` contiene exactamente los 27 paquetes reales. La diferencia numérica es pequeña (72,5 frente a 72,7 %), pero una evidencia de cobertura que incluye código inexistente no es evidencia válida aunque el porcentaje salga parecido.

Clase de prueba	Nº	Qué verifica
<code>AuthServiceTest</code>	5	Login correcto e incorrecto, registro, registro duplicado, disponibilidad.
<code>JwtServiceTest</code>	6	Emisor y audiencia válidos, audiencia distinta rechazada, token manipulado, refresco.
<code>LoginAttemptServiceTest</code>	6	Bloqueo al alcanzar el límite, no reinicio del TTL, borrado del contador al acertar.
<code>RedisBlacklistServiceTest</code>	6	Revocación con TTL restante, TTL no positivo ignorado, consulta de revocación.
<code>EstudianteControllerTest</code>	9	Los siete verbos del recurso más 404 y 422.
<code>EstudianteServiceTest</code>	11	Paginación envuelta, baja lógica, persistencia transaccional, delegación al procedimiento SQL.
<code>BackendApplicationTests</code>	1	Arranque del contexto contra el esquema migrado.
<code>CategoriaServiceTest</code>	9	Paginación, alta, edición, baja lógica, validación de rango de edad.
<code>CategoriaControllerTest</code>	7	Mapeo HTTP: 200/201/204/400/404/422.
<code>EntrenadorServiceTest</code>	6	Paginación con mapeo persona/usuario, persona o usuario duplicados, alta, baja lógica.
<code>EntrenadorControllerTest</code>	5	Mapeo HTTP: 200/201/204/404/422.
<code>UsuarioServiceTest</code>	6	Paginación, username duplicado, alta con contraseña codificada, persona inexistente, baja lógica.
<code>UsuarioControllerTest</code>	6	Mapeo HTTP: 200/201/204/400/422.
<code>PersonaServiceTest</code>	8	Paginación, búsqueda por cédula, unicidad de cédula/correo al crear y editar, baja lógica.
<code>PersonaControllerTest</code>	6	Mapeo HTTP: 200/201/204/400/422.
<code>EstadoGeneralServiceTest</code>	2	Listado completo y listado vacío.
<code>EstadoGeneralControllerTest</code>	1	Mapeo HTTP del único <i>endpoint</i> .
Total	101	

Un detalle técnico real que surgió al escribir estas pruebas: los cuatro controladores que reciben `Pageable` como parámetro (`Categoria`, `Entrenador`, `Usuario`, `Persona`) fallaban con 500 bajo `MockMvcBuilders.standaloneSetup()`, porque ese modo de arranque no registra el `PageableHandlerMethodArgumentResolver` que sí provee el contexto completo de Spring — `EstudianteController` no lo sufre porque usa `@RequestParam` planos en vez de `Pageable`. Se resolvió registrando el resolutor explícitamente en cada prueba.

Un detalle relevante de una entrega anterior: se había descubierto que JaCoCo *nunca* había generado cobertura real, porque el `argLine` configurado en Surefire sobrescribía el `javaagent` del propio JaCoCo. Ese defecto está corregido; por eso esta medición pudo mostrar honestamente primero una regresión y después la mejora real.

Las pruebas de `LoginAttemptServiceTest` merecen mención aparte: verifican no solo que el bloqueo ocurra, sino que un fallo posterior *no* reinicie la ventana de tiempo. Sin esa segunda propiedad, un atacante podría mantener la cuenta en estado de bloqueo indefinido, o bien —según cómo se implemente— reiniciar el contador y evadir el límite.

7.4. Calidad web: Lighthouse (Bloque C.5)

Tres corridas con perfil móvil (Lighthouse v13.4.1):

Categoría	Media	Umbral	Estado
Rendimiento	92,3	≥ 80	OK
Accesibilidad	100	≥ 90	OK
Buenas prácticas	96	≥ 90	OK
SEO	63	≥ 90	Ver §7.4.1

Métricas Web Vitals: LCP 2,86 s; FCP 2,43 s; TBT 25,7 ms; y CLS **0,000** en las tres corridas (la interfaz no desplaza contenido durante la carga).

Esta medición expuso, y permitió corregir, tres defectos reales del frontend: el documento declaraba `lang="en"` en una aplicación íntegramente en español; conservaba el título `Frontend` generado por Angular CLI y no tenía descripción; y carecía de un elemento `<main>`, lo que hacía fallar la auditoría de puntos de referencia de accesibilidad. Corregidos los tres, la accesibilidad pasó de 97 a **100**.

Se corrigió además un defecto en la propia configuración de medición: `lighthouse.js` declaraba `preset: 'mobile'`, un valor que no existe en Lighthouse, lo que abortaba toda corrida con código de salida 1. Es decir, **esta medición nunca se había podido ejecutar**.

7.4.1. Por qué el SEO de 63 es el resultado correcto

El puntaje SEO bajó de 82 a 63 *a causa de una corrección deliberada*. Al añadir un `robots.txt` válido con `Disallow: /`, Lighthouse activa la penalización `is-crawlable` («página bloqueada para indexación»), que por sí sola descuenta 27 puntos, porque su categoría SEO asume que el sitio *quiere* ser indexado.

SGED es una aplicación de gestión interna que trata datos personales de menores de edad. Que fuese indexable por buscadores sería un defecto de privacidad, no una virtud. Elevar el SEO a 90 exigiría eliminar el `robots.txt`: degradar la privacidad del sistema para mejorar una métrica. Se optó por lo contrario, se relajó el umbral agregado de esa categoría a advertencia con la justificación escrita en el propio archivo de configuración, y se mantuvieron como bloqueantes las auditorías SEO que sí aplican a una aplicación interna (`meta-description`, `document-title`, `html-has-lang`, `viewport`), todas ellas en 1,0. **OK**

7.5. Usabilidad: SUS (Bloque C.3)

El instrumento (`docs/mediciones/sus/INSTRUMENTO-SUS.md`) es el cuestionario SUS de diez ítems [4] con la escala adjetival de interpretación [3]. El script `scripts/sus-analysis.py` calcula media, desviación típica, intervalo de confianza al 95 %, mediana y grado por participante; se niega deliberadamente a ejecutarse con menos de diez participantes, y su aritmética quedó verificada contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos) antes de usarlo con datos reales.

Aplicado el 2026-07-30 a 10 participantes (formularios en papel, transcritos a `docs/mediciones/sus/respuestas` sin registrar nombres, solo un código de participante y su perfil): 3 entrenadores, 3 recepcionistas, 2 estudiantes, 2 padres de familia.

Métrica	Valor
Media SUS	68,25
Desviación típica	22,14
IC 95 %	$68,25 \pm 13,72$ (54,53 – 81,97)
Mediana	73,75
Grado	C — Aceptable

La media cruza el umbral de 68 por apenas 0,25 puntos, y el intervalo de confianza es ancho: con esta muestra no puede afirmarse con confianza estadística que la media poblacional real supere 68, solo que 68,25 es la mejor estimación puntual disponible. No se presenta como un resultado cómodo.

El hallazgo más informativo no es la media, es su distribución bimodal por perfil, que la agregación esconde:

Perfil	Promedio SUS	Grado
Entrenador (n=3)	86,7	A
Estudiante (n=2)	90,0	A
Recepcionista (n=3)	51,7	F
Padre de familia (n=2)	43,75	F

Entrenadores y estudiantes calificaron el sistema como excelente; recepcionistas y padres de familia, como inaceptable. El CRUD de estudiantes implementado hasta esta entrega (RF-08 a RF-15) es exactamente el flujo operado por un rol administrativo — no el de un entrenador consultando información, ni el de un padre buscando visibilidad sobre su representado —, lo que sugiere que la interfaz actual está más pulida para un caso de uso que no es el de quienes peor la calificaron. Es una pista concreta de dónde enfocar el trabajo de usabilidad antes de la Entrega Final, no solo un número que aprobó por poco.

8. Reproducibilidad

El sistema se levanta completo desde una clonación limpia con un solo comando:

```
1 git clone https://github.com/DarwinSM21/SGED_APPWEB.git
2 cd SGED_APPWEB
3 cp .env.example .env
4 make up
```

Listing 4: Arranque reproducible

Objetivo	Acción
<code>make up</code>	Levanta el sistema completo
<code>make test</code>	JUnit 5 + cobertura JaCoCo
<code>make bench</code>	3 corridas k6 + análisis con IC 95 %
<code>make audit</code>	Auditoría OWASP (6 controles) + SQL dinámico
<code>make clean</code>	Limpia contenedores, volúmenes y compilaciones

Las imágenes de PostgreSQL y Redis están fijadas por digest SHA-256, no por etiqueta móvil: dos construcciones del mismo *commit* usan exactamente los mismos binarios. La reproducibilidad se verificó clonando el repositorio en un directorio independiente, no solo reiniciando el volumen local.

9. Consideraciones éticas

El sistema trata datos personales de niños, niñas y adolescentes: nombres, cédula, fecha de nacimiento, asistencia con hora y lugar, y evaluaciones individuales de desempeño. Esa combinación es sensible porque los titulares son menores que no pueden consentir por sí mismos, porque la asistencia es información de localización temporal, y porque las evaluaciones constituyen perfilado de personas [1, 2].

Se aplicó minimización: se decidió *no* recolectar peso, altura, contextura ni datos de alimentación, pese a haber sido considerados en el diseño, por tratarse de datos de salud cuya recolección exigiría garantías que este proyecto no puede sostener.

`docs/etica/ETHICS.md` documenta además cinco hallazgos abiertos, en lugar de afirmar que todo está resuelto: la cédula se almacena sin validación ni cifrado; las observaciones del entrenador son texto libre sin control sobre un menor; no existe mecanismo de supresión de datos (la baja lógica preserva el historial pero no es un borrado); el consentimiento del representante no está modelado, lo que bloquea el módulo de notificaciones; y el certificado TLS es autofirmado, adecuado para evaluación pero no para producción.

Todos los datos del repositorio son ficticios. No se utilizaron datos reales de menores en ninguna etapa del desarrollo ni de las mediciones.

10. Trazabilidad y honestidad académica

La matriz `docs/trazabilidad/matriz.csv` enlaza cada requisito con su historia de usuario, su caso de uso, su *endpoint*, el archivo que lo implementa, la prueba JUnit que lo cubre y la evidencia empírica correspondiente.

El historial de `git` evidencia la autoría de cada integrante. Se documenta en `CONTRIBUTORS.md` que dos *commits* figuran bajo una cuenta distinta por haber sido realizados desde un equipo de la universidad con otra sesión configurada; la aclaración se hace por escrito en lugar de reescribir el historial ya compartido. El mismo archivo declara explícitamente el uso de herramientas de asistencia por inteligencia artificial, siguiendo la guía de transparencia del SWEBOK v4.0 [14].

11. Conclusiones

1. El sistema cumple con holgura los umbrales cuantitativos exigidos: p95 de 14,18 ms frente a un límite de 200 ms, 6 de 6 controles OWASP verificados con evidencia reproducible, accesibilidad Lighthouse de 100/100, y **72,7 %** de cobertura de pruebas frente a un mínimo de 60 % (sección 7.3) — cifra alcanzada después de detectar y corregir una regresión real a 39,8 % causada por los recursos nuevos de la reestructuración, no reportando directamente un número cómodo.

2. El aporte más sustantivo de esta entrega no fue añadir funcionalidad, sino *verificar* lo que ya existía, dos veces: una vez contra el código de la entrega original, y una segunda vez después de que otros dos integrantes reestructuraran el backend en paralelo. Ambas rondas revelaron defectos que ninguna revisión superficial había detectado — desde que JaCoCo no medía cobertura en absoluto, hasta que un ADR de arquitectura describía un mecanismo de autenticación (JWT en `localStorage`) distinto del que el código realmente usa (cookie `HttpOnly`). Todos los defectos encontrados se corrigieron y quedan documentados en la sección 3 en vez de ocultarse.
3. La distinción entre lo implementado y lo modelado se mantiene explícita en todo el documento. En el caso de **Representante** y **Equipo** se llevó un paso más allá: en vez de dejar catorce clases vacías —una de ellas de cero bytes, el mismo defecto que motivó tres observaciones del docente en la Entrega 1B— se eliminaron del código y ambos módulos quedan declarados como pendientes solo en la documentación (sección 3). Un archivo vacío no cuenta como funcionalidad entregada, y tampoco debería quedarse en el repositorio aparentando que sí.
4. La encuesta SUS (sección 7.5) ya no es un frente abierto, pero su resultado abre uno nuevo: el sistema es excelente para entrenadores y estudiantes (grado A) e inaceptable para receptionistas y padres de familia (grado F). Mejorar la usabilidad del flujo administrativo y del futuro módulo de representantes es, con evidencia real detrás, más urgente que subir la media agregada. La exposición REST del dominio deportivo restante (horarios, sesiones, asistencias, evaluaciones) sigue siendo el objetivo natural de la Entrega Final.

Referencias

- [1] Asamblea Nacional del Ecuador. Ley orgánica de protección de datos personales. Registro Oficial Suplemento 459, 26 de mayo de 2021, 2021.
- [2] Association for Computing Machinery. ACM code of ethics and professional conduct. <https://www.acm.org/code-of-ethics>, 2018.
- [3] Aaron Bangor, Philip Kortum, and James Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009.
- [4] John Brooke. SUS: A ‘quick and dirty’ usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [5] Simon Brown. *Software Architecture for Developers, Volume 2: Visualise, document and explore your software architecture*. Leanpub, 2018. Modelo C4.
- [6] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models, 2011.
- [7] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 — systems and software engineering — life cycle processes — requirements engineering, 2018.
- [8] M. B. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). Request for Comments 7519, Internet Engineering Task Force (IETF), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [9] M. Nottingham, E. Wilde, and S. Dalal. Problem details for HTTP APIs. Request for Comments 9457, Internet Engineering Task Force (IETF), July 2023. <https://www.rfc-editor.org/rfc/rfc9457>.
- [10] Michael T. Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011.
- [11] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. <https://owasp.org/Top10/>, 2021.
- [12] PostgreSQL Global Development Group. PostgreSQL 16 documentation — CREATE PROCEDURE. <https://www.postgresql.org/docs/16/sql-createprocedure.html>, 2026.
- [13] Redgate. Flyway documentation — migrations. <https://documentation.red-gate.com/flyway/>, 2026.
- [14] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.